250226

February 26, 2026

0.0.1 Imports

[2]:
```
## Nessacery imports for entire lab done here for easy management
from scipy.optimize import minimize
import numpy as np
import matplotlib.pyplot as plt
import hyperspy.api as hs
from scipy import optimize
```

0.0.2 Functions

[58]:
```
## Defining perodic function with a sine and cosine term, variables allow␣
↪manipulation of all characteristics,
## while the presence of a sin and cos allow variation of amplitude etc in a␣
↪period
def perodic(x,a,b,c,d,e,f,g):
    return (a*np.sin(b*np.deg2rad(x) + c) + (d*np.cos(e*np.deg2rad(x) + f)) + g)


## Definig perodic plot function to fit the perodic function to one slice of␣
↪data
def periodic_fit_plot(x_variable, y_variable,initial, plot = False):
    bounds = (
    [0,0,0,0,0,0,0],
    [1000,4,180,1000,4,180,1000])
## Using optimize.curve_fit to fit a perodic funtion to our x and y data,␣
↪intial values for parameters taken from input
    params_sin, params_sin_covariance = optimize.
↪curve_fit(perodic,x_variable,y_variable,p0=initial,bounds=bounds,maxfev=30000)

## Finding estimated uncertainties for each parameter
    errs = np.sqrt(np.diag(params_sin_covariance))

    ## Rounding frequencies, ensuring single values at 0/360
    params_sin[1] = np.round(params_sin[1], decimals=0)
    params_sin[4] = np.round(params_sin[4], decimals=0)
    return(params_sin,errs)
```

[14]:
```python
def periodic_fit_plot_whole(dataset, initial, plot=False):
    import matplotlib.pyplot as plt
    import matplotlib.gridspec as gridspec
    import numpy as np
    from tqdm import tqdm

    params_array = []
    errors_array = []

    degrees = np.linspace(0, 360, 360)

    nx = len(dataset[:, 0, 0])
    ny = len(dataset[0, :, 0])

    total_iterations = nx * ny

    with tqdm(total=total_iterations, desc="Processing slices") as pbar:
        for x in range(nx):
            for y in range(ny):
                params = periodic_fit_plot(
                    degrees,
                    dataset[x, y, :],
                    initial,
                    plot=False
                )
                params_array.append(params[0])
                errors_array.append(params[1])
                pbar.update(1)

    params = np.array(params_array)
    errors = np.array(errors_array)

    params_reshaped = params.reshape(nx, ny, 7)
    errors_reshaped = errors.reshape(nx, ny, 7)

    return params_reshaped, errors_reshaped
```

[5]:
```python
def decomp_data(path, components, algo):
    raw_data = hs.load(path)
    raw_data_end = np.load(path)
    data_hs_func = hs.signals.Signal1D(raw_data.data)
    data_hs_func.decomposition(algorithm=algo)
    sc = data_hs_func.get_decomposition_model(components)
    decomposed_data = sc.data

    return decomposed_data, raw_data_end
```

[6]:
```
def decomp_and_fit(path, components, algo):
    decomp = decomp_data(path,components,algo)

    decomp_params =␣
 ↪periodic_fit_plot(x,decomp[0][0][20],[20,1,1,20,1,1,5],plot=False)
    params = periodic_fit_plot(x,decomp[1][0][20],[20,1,1,20,1,1,5],plot=False)

    return decomp[0], decomp[1], decomp_params, params
```

[7]:
```
def find_elbow(varience):
    """
    Find elbow point in scree plot data.

    Parameters:
        varience : array - Varience explained values

    Returns:
        elbow_x : component at elbow
        elbow_index : index of elbow
        relevant_components : components to keep
    """

    x = np.linspace(1,len(varience),len(varience))
    y = np.array(varience)

    # Normalize data (important for generic datasets)
    x_norm = (x - x.min()) / (x.max() - x.min())
    y_norm = (y - y.min()) / (y.max() - y.min())

    # Line between first and last points
    line_vec = np.array([x_norm[-1] - x_norm[0],
                         y_norm[-1] - y_norm[0]])
    line_vec = line_vec / np.linalg.norm(line_vec)

    # Compute distance from each point to the line
    distances = []
    for i in range(len(x_norm)):
        point_vec = np.array([x_norm[i] - x_norm[0],
                              y_norm[i] - y_norm[0]])

        proj_len = np.dot(point_vec, line_vec)
        proj_vec = proj_len * line_vec

        dist_vec = point_vec - proj_vec
        distances.append(np.linalg.norm(dist_vec))

    elbow_index = np.argmax(distances)
```

```
    elbow_x = x[elbow_index]

    relevant_components = x[:elbow_index + 1]

    return elbow_x, elbow_index, relevant_components
```

[8]:
```
## Adjusted R square
import numpy as np

def adjusted_r2(y_true, y_pred, n_params):
    """
    Compute adjusted R-squared for a model.

    Parameters
    ----------
    y_true : array - True Data
    y_pred : array - Fit Predictions
    n_params : int - Number of parameters in fit

    Returns
    -------
    r2_adj : float - Adjusted R-squared.
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    n = len(y_true)

    # Residual sum of squares
    ss_res = np.sum((y_true - y_pred) ** 2)

    # Total sum of squares
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)

    # Standard R^2
    r2 = 1 - ss_res / ss_tot

    # Adjusted R^2
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - n_params - 1)

    return r2_adj
```

0.0.3 Data Processing

[9]:
```
LSAT_data = np.load("binned_diff_20210421_162329_fast_2026_PT1D.npy")
```

[ ]:
```
## Testing the function with a cropped dataset
```

[41]:
```
LSAT_cropped = LSAT_data[0:10,0:10,:]
np.save("LSAT_Cropped.npy", LSAT_cropped)
```

[44]:
```
sd = decomp_data("LSAT_Cropped.npy",3,"SVD")
```

```
Decomposition info:
  normalize_poissonian_noise=False
  algorithm=SVD
  output_dimension=None
  centre=None
```

[59]:
```
decomped_params_data, decomped_errors_data =␣
↪periodic_fit_plot_whole(sd[0],[60,1,1,60,1,1,1])
```

```
Processing slices: 100%|      | 100/100 [00:23<00:00,  4.18it/s]
```

[18]:
```
np.save("LSAT_Params_Decomped.npy", decomped_params_data)
np.save("LSAT_Errors_Decomped.npy", decomped_errors_data)
```

[60]:
```
raw_params_data, raw_errors_data =␣
↪periodic_fit_plot_whole(sd[1],[60,1,1,60,1,1,1])
```

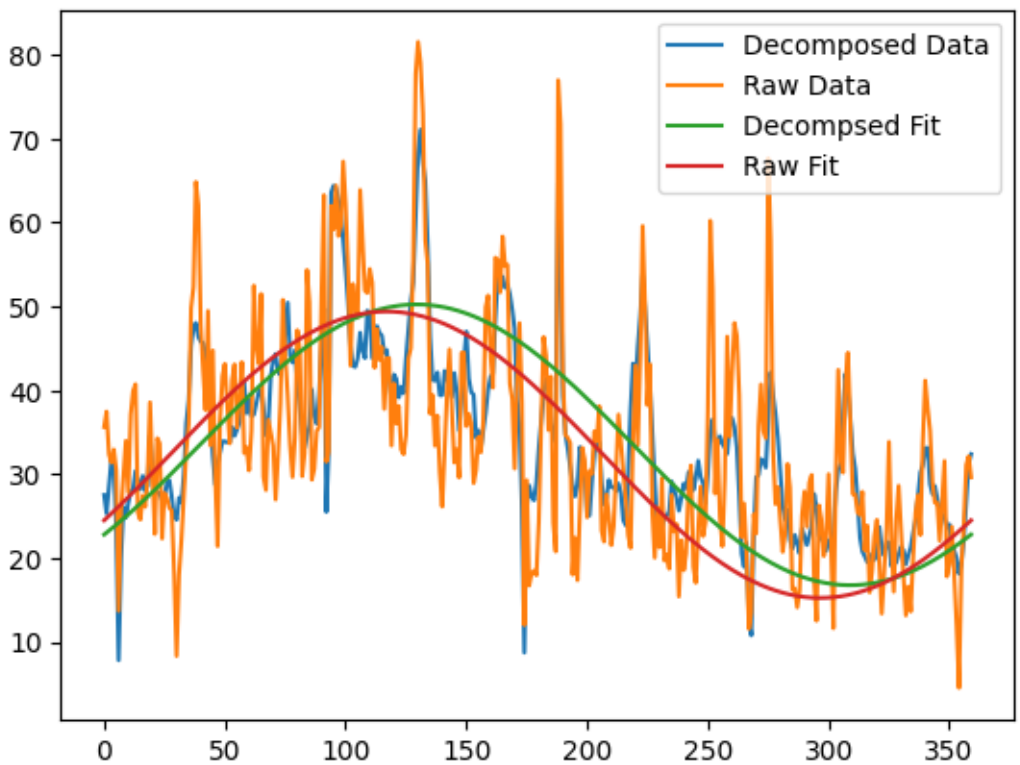
```
Processing slices: 100%|      | 100/100 [00:29<00:00,  3.34it/s]
```

[21]:
```
np.save("LSAT_Params_Raw.npy", raw_params_data)
np.save("LSAT_Errors_Raw.npy", raw_errors_data)
```

[96]:
```
## Fits still look good after bounding parameters
```

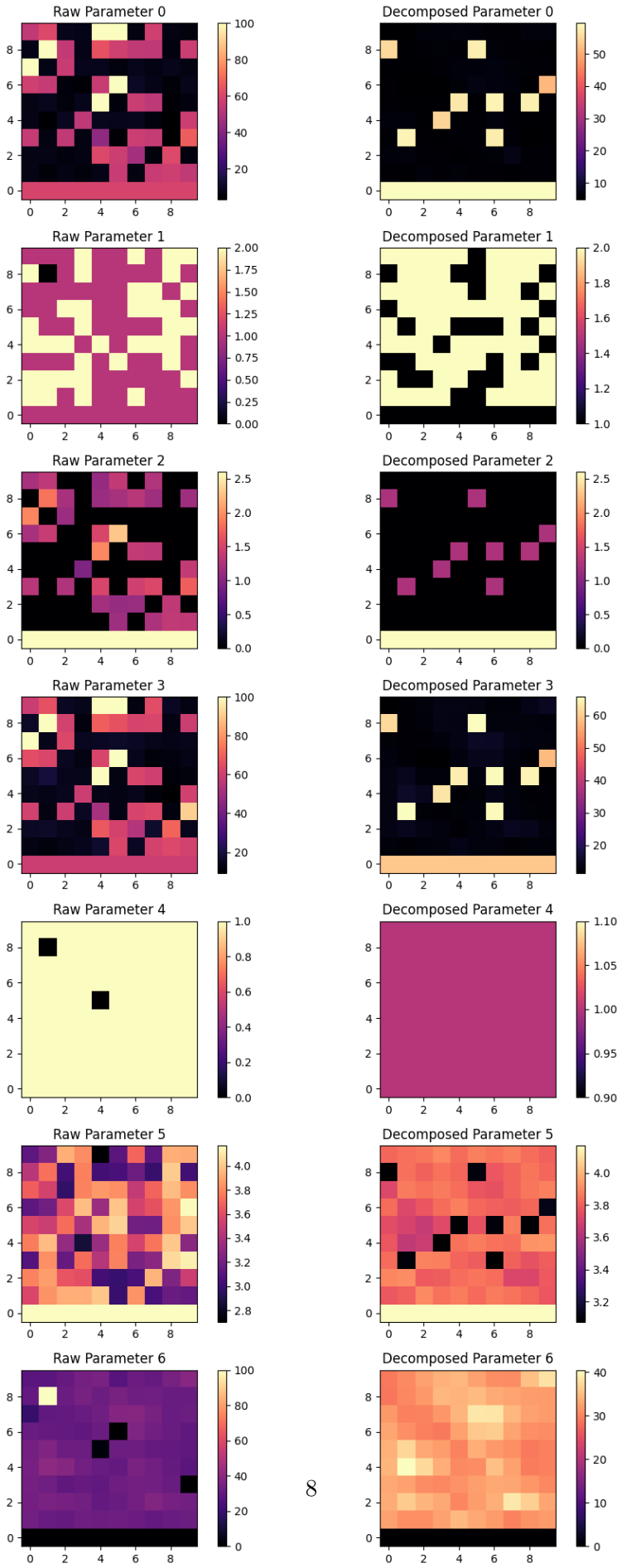
[97]:
```
x = np.linspace(0,360,360)

plt.plot(sd[0][5][5],label="Decomposed Data")
plt.plot(sd[1][5][5],label="Raw Data")
plt.plot(perodic(x,*decomped_params_data[5][5]),label="Decompsed Fit")
plt.plot(perodic(x,*raw_params_data[5][5]),label="Raw Fit")
plt.legend()
```

[97]:
```
<matplotlib.legend.Legend at 0x7f163fff43a0>
```

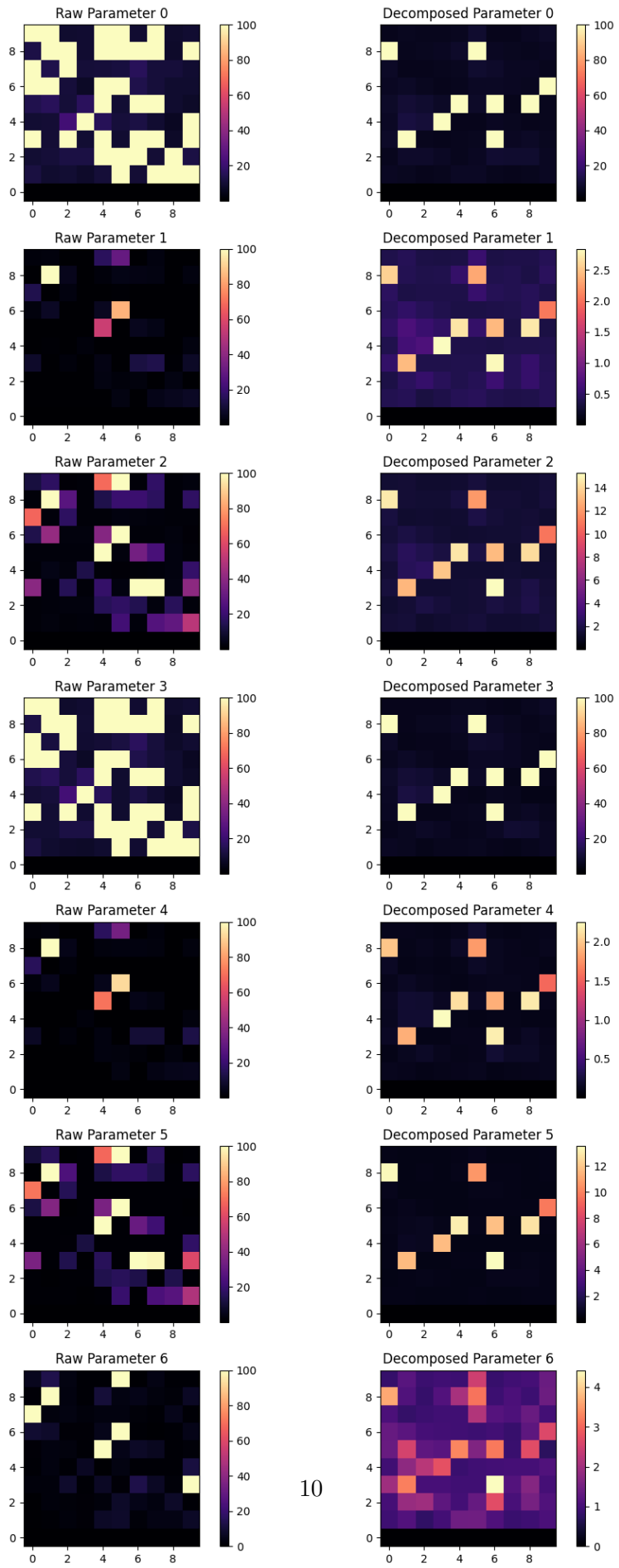


[100]:
```
## new function to visualise parameters (or errors) for a raw and decomped 2d␣
↪scan
def visualise_params(raw,decomped,clip):
    fig, axes = plt.subplots(7, 2,figsize=(10,20))

    for n in range(7):

    # Raw
        im0 = axes[n, 0].imshow(np.clip(raw[:, :, n], 0,␣
↪clip),cmap='magma',origin='lower')
        axes[n, 0].set_title(f'Raw Parameter {n}')
        fig.colorbar(im0, ax=axes[n, 0])

    # Decomposed
        im1 = axes[n, 1].imshow(np.clip(decomped[:, :, n], 0,␣
↪clip),cmap='magma',origin='lower')
        axes[n, 1].set_title(f'Decomposed Parameter {n}')
        fig.colorbar(im1, ax=axes[n, 1])

    plt.tight_layout()
    plt.show()
```

[101]:
```
visualise_params(raw_params_data,decomped_params_data,100)
```


Raw Parameter 0
Decomposed Parameter 0
Raw Parameter 1
Decomposed Parameter 1
Raw Parameter 2
Decomposed Parameter 2
Raw Parameter 3
Decomposed Parameter 3
Raw Parameter 4
Decomposed Parameter 4
Raw Parameter 5
Decomposed Parameter 5
Raw Parameter 6
Decomposed Parameter 6

[102]:
```
visualise_params(raw_errors_data,decomped_errors_data,100)
```


Raw Parameter 0
Decomposed Parameter 0
Raw Parameter 1
Decomposed Parameter 1
Raw Parameter 2
Decomposed Parameter 2
Raw Parameter 3
Decomposed Parameter 3
Raw Parameter 4
Decomposed Parameter 4
Raw Parameter 5
Decomposed Parameter 5
Raw Parameter 6
Decomposed Parameter 6

[103]:
```
## Everything seems to be working, can move onto applying this to the full data␣
↪set
```